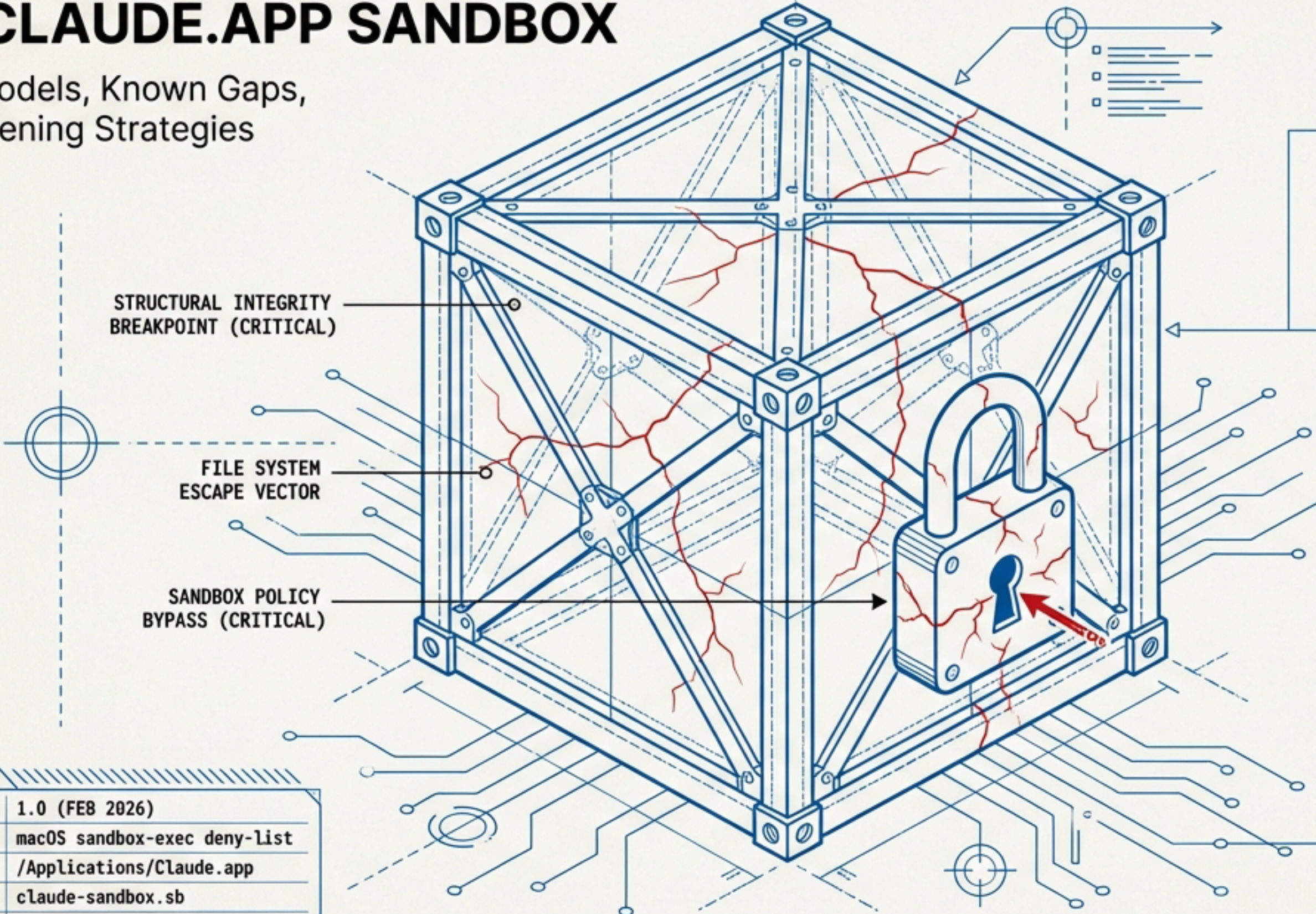


SECURITY ANALYSIS: THE CLAUDE.APP SANDBOX

Threat Models, Known Gaps,
and Hardening Strategies



HARDENING STRATEGIES

- [SOLUTION] - Implement Strict 'deny-list' with Explicit Allow-Listing - [REDACTED]
- [SOLUTION] - Utilize 'mach-lookup' Restrictions for XPC Services - [REDACTED]
- [SOLUTION] - Enable Mandatory Access Control (MAC) Policies - [REDACTED]
- [SOLUTION] - Conduct Comprehensive Fuzzing of 'claude-sandbox.sb' - [REDACTED]
- [SOLUTION] - Deploy Kernel Integrity Protection (KIP) Measures - [REDACTED]

THREAT MODELS & RISKS

KNOWN GAPS

- ⚠ [CRITICAL] - /System/Library/PrivateFrameworks/TCC.framework/Resources/tecutil READ ACCESS - [REDACTED]
- ⚠ [HIGH] - com.apple.security.app-sandbox-read-write-files ALLOWLISTING - [REDACTED]
- ⚠ [HIGH] - DYLD_INSERT_LIBRARIES INJECTION PATH - [REDACTED]

ATTACK VECTORS

- ⚠ [CRITICAL] - Arbitrary Code Execution via ROP Chains in 'claude.sb' - [REDACTED]
- ⚠ [HIGH] - Data Exfiltration through XPC Inter-Process Communication - [REDACTED]
- ⚠ [CRITICAL] - Kernel-Level Privilege Escalation (KASLR Bypass) - [REDACTED]

META-DATA	
DOCUMENT VER:	1.0 (FEB 2026)
TARGET:	macOS sandbox-exec deny-list
SCOPE:	/Applications/Claude.app
CONFIG:	claude-sandbox.sb
STATUS:	VULNERABILITY ASSESSMENT



THE PRAGMATIC COMPROMISE

USABILITY

Electron App Functions 
Full API Access 
User Experience 



SECURITY

 File System Deny-List
 Anti-Delete Rules

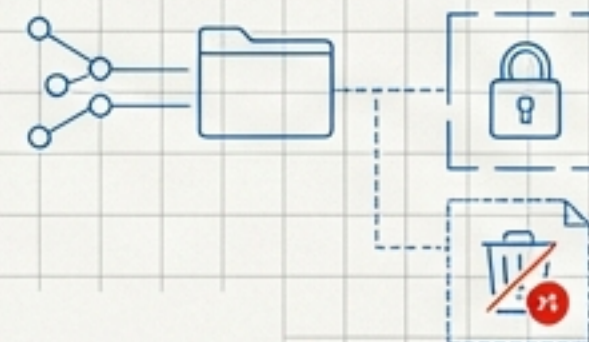
**“The sandbox protects your data from Claude,
but it does not protect your project from Claude.”**

WHAT IT DOES

- ✓ Prevents accidental mass deletion (`rm -rf`).
- ✓ Blocks known-sensitive paths like `~/Documents` and `~/Library`.

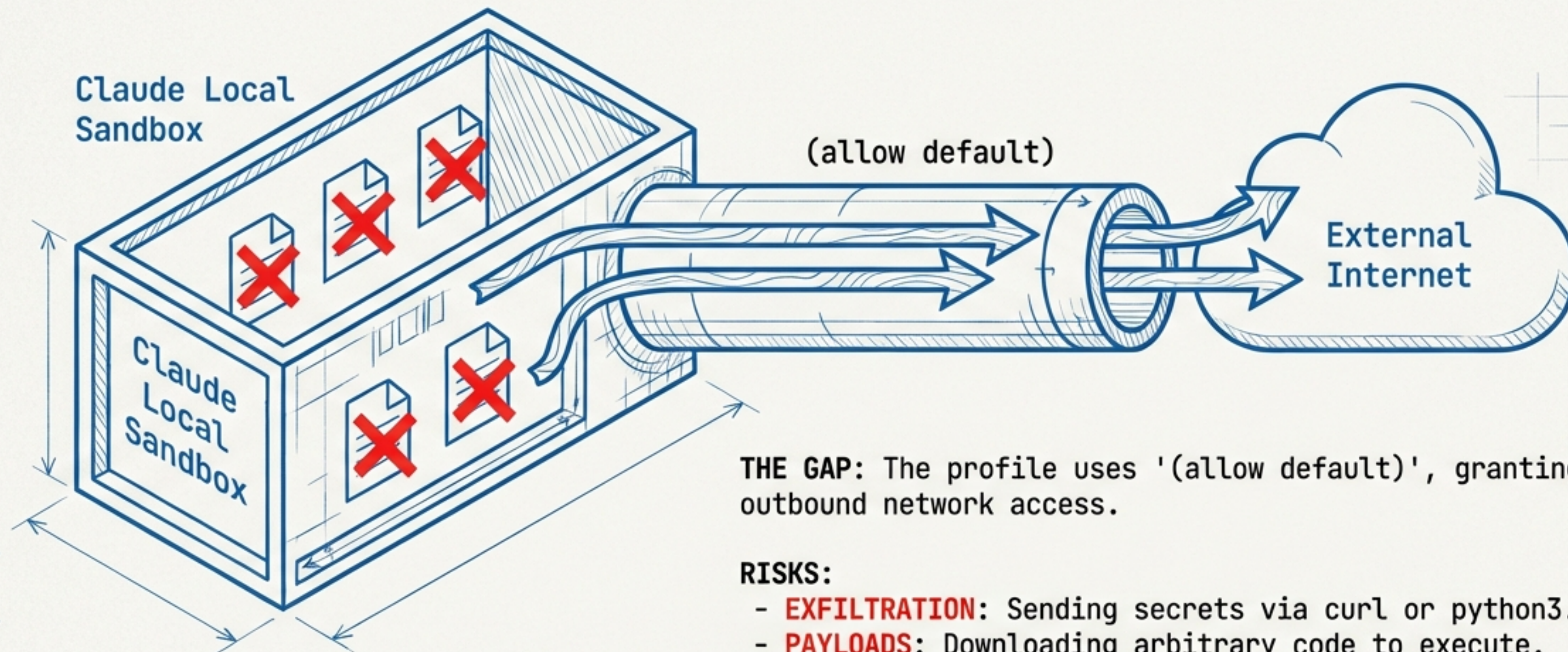
WHAT IT MISSES

- ! Sophisticated, deliberate attacks.
- ! Malicious intent via prompt injection.
- ! Access to non-standard paths.



SEVERITY: HIGH

THE OPEN DOOR: UNRESTRICTED NETWORK



THE GAP: The profile uses '(allow default)', granting full outbound network access.

RISKS:

- **EXFILTRATION:** Sending secrets via curl or python3.
- **PAYLOADS:** Downloading arbitrary code to execute.
- **REVERSE SHELLS:** Establishing persistent remote control.

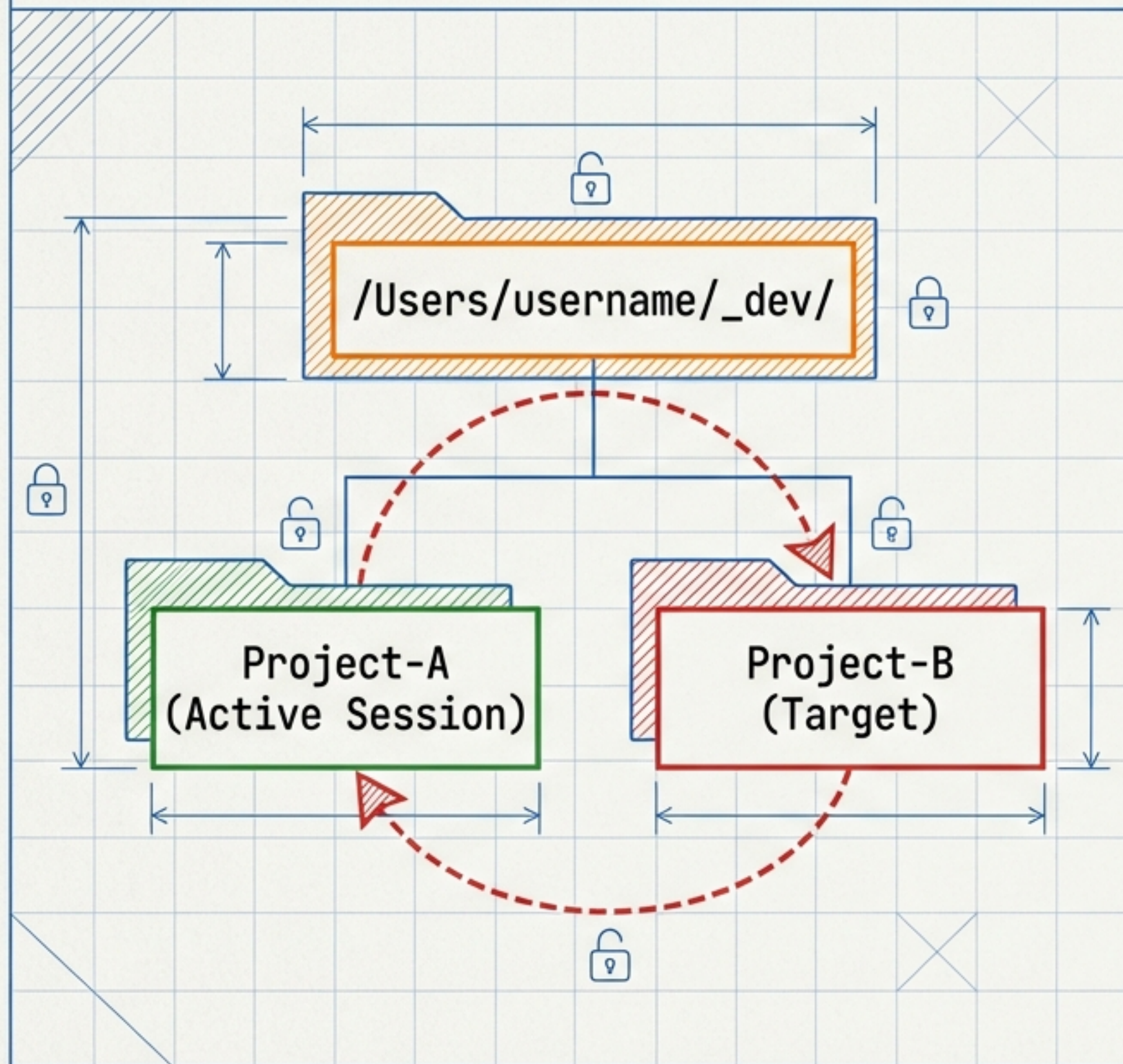
CONSTRAINT:

Blocking 'network-outbound' breaks the Anthropic API. External firewalls are required.

NAME	
DATE	
TIME	
BY	

LATERAL MOVEMENT: SIBLING PROJECTS

SEVERITY: HIGH



THE GAP:

The deny list protects personal folders (Documents) but misses sibling directories in dev workspaces.

PATH:

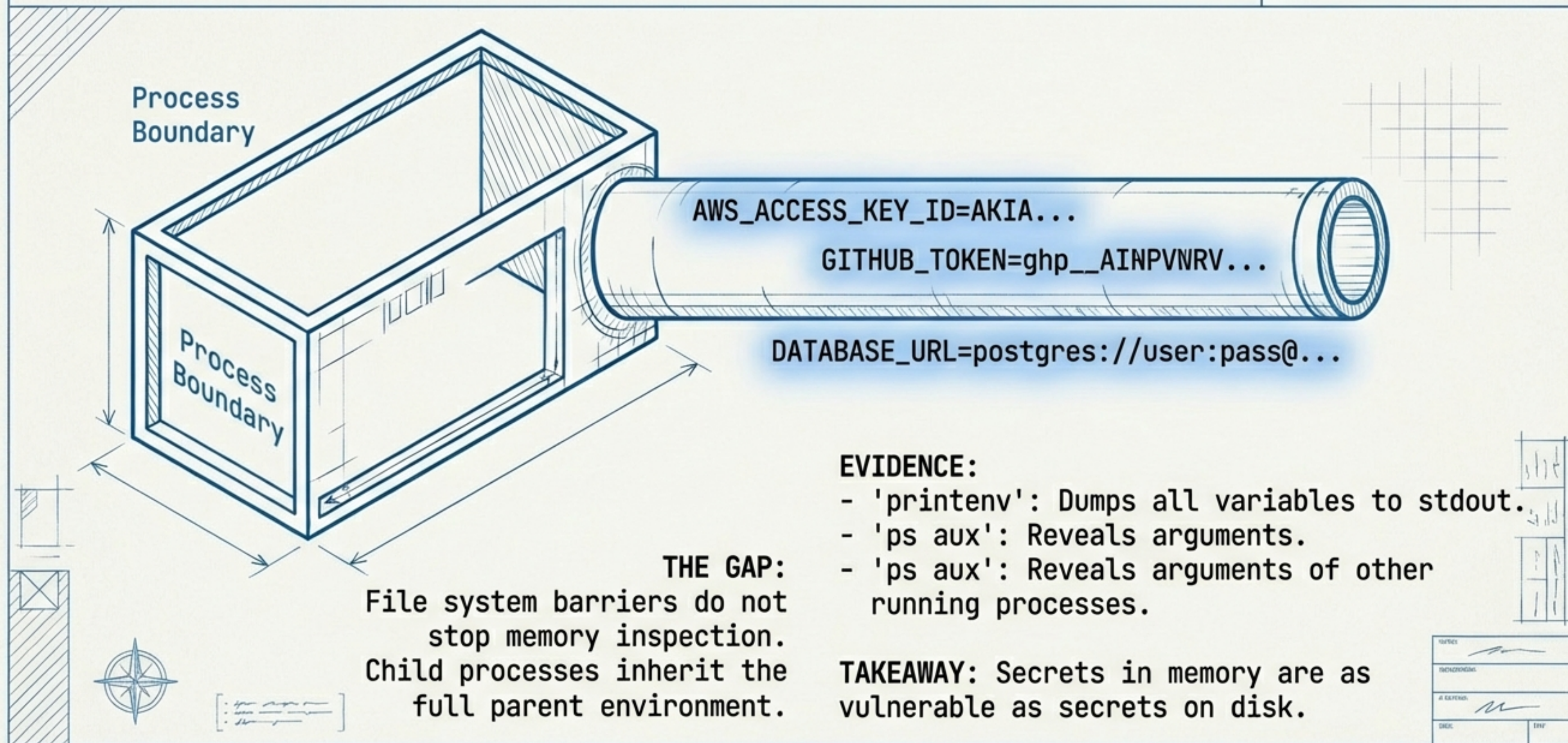
`/Users/username/_dev/` (Recursive Read/Write)

ATTACK:

- Reading source code and `.env` files from inactive projects.
- Supply Chain Injection: Modifying code in adjacent repositories.
- Accessing global `.gitconfig` credentials.

INVISIBLE LEAKS: ENVIRONMENT & PROCESS

SEVERITY: HIGH



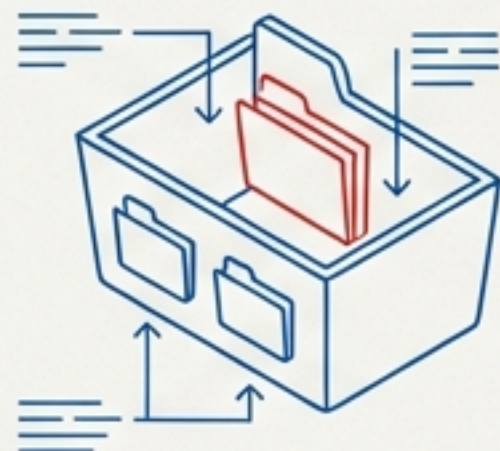
THE LONG TAIL OF ACCESS: BLIND SPOTS

A deny-list is inherently incomplete; anything forgotten is allowed.

SEVERITY: HIGH

TEMP FILES

Shared /tmp and /var/folders allow reading data from other apps.



CLIPBOARD

'pbpaste' access allows monitoring copied passwords or tokens.



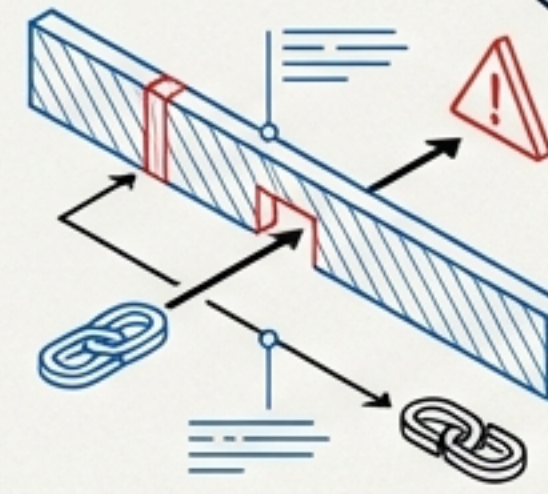
CLOUD CREDENTIALS

~/.kube/config, ~/.aws/, ~/.azure/ often missed by default deny rules.



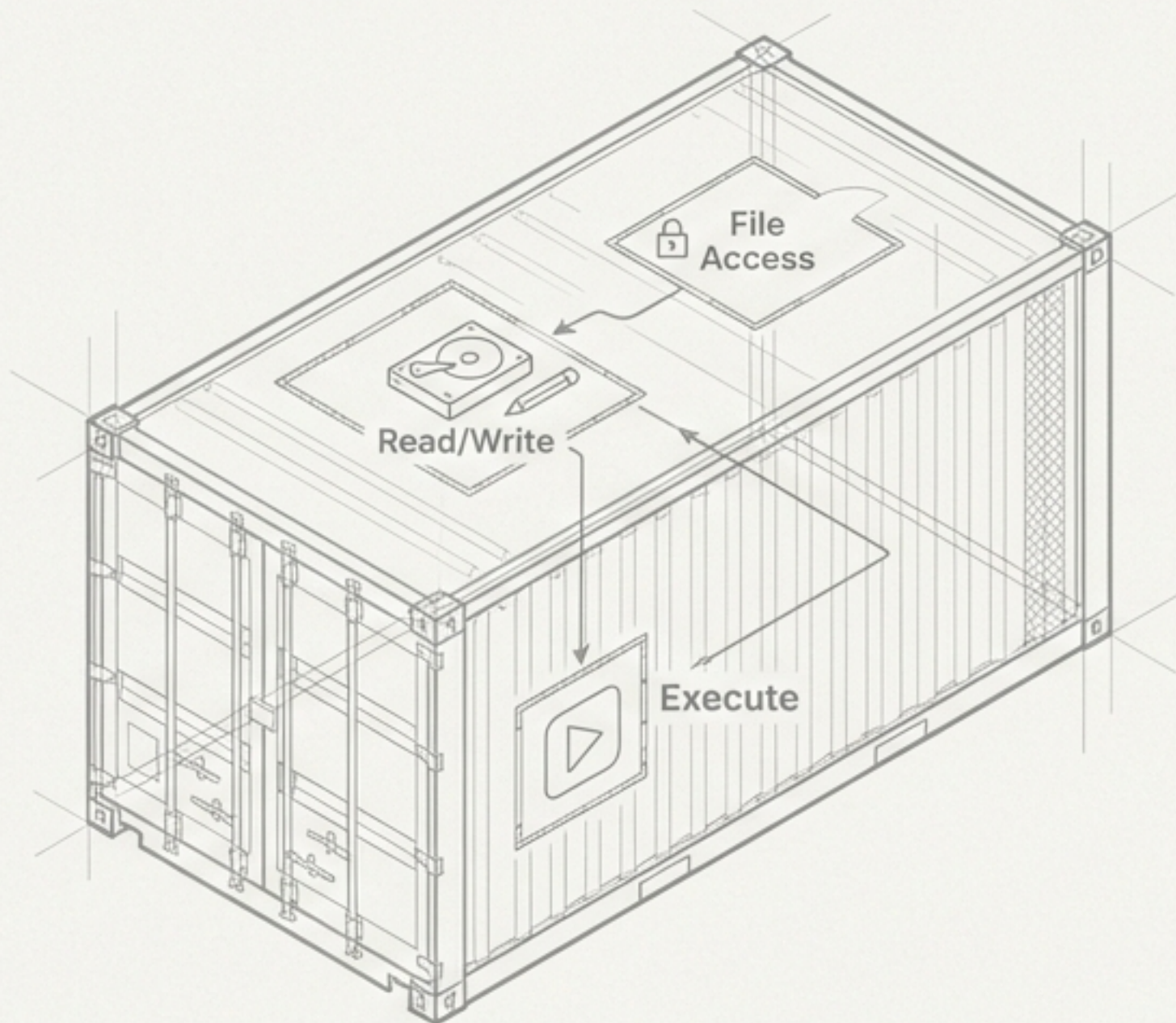
SYMLINKS

Potential to bypass path-based rules via symbolic link resolution.

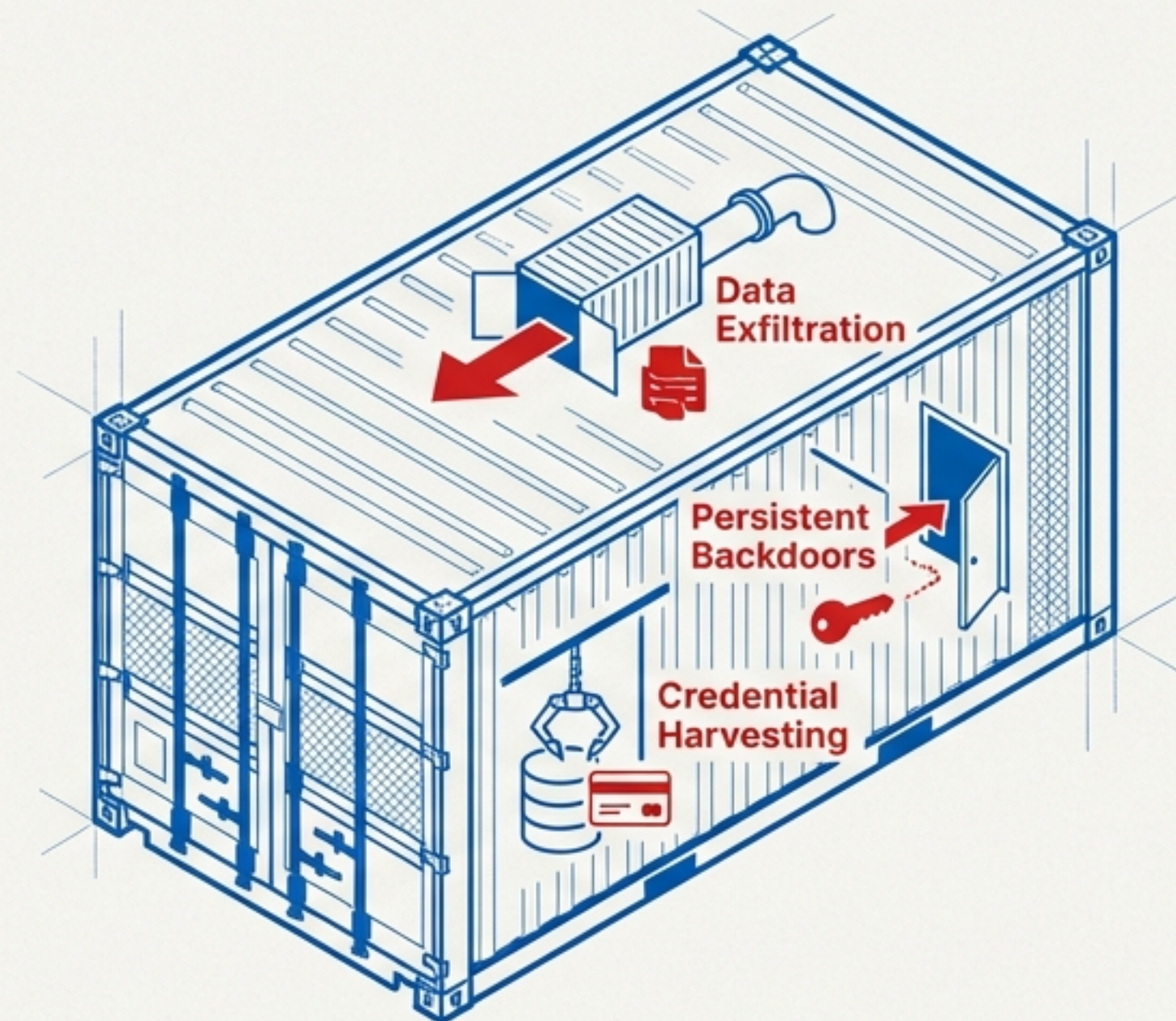


THE NEW THREAT MODEL: MISALIGNMENT

STATIC CAPABILITY



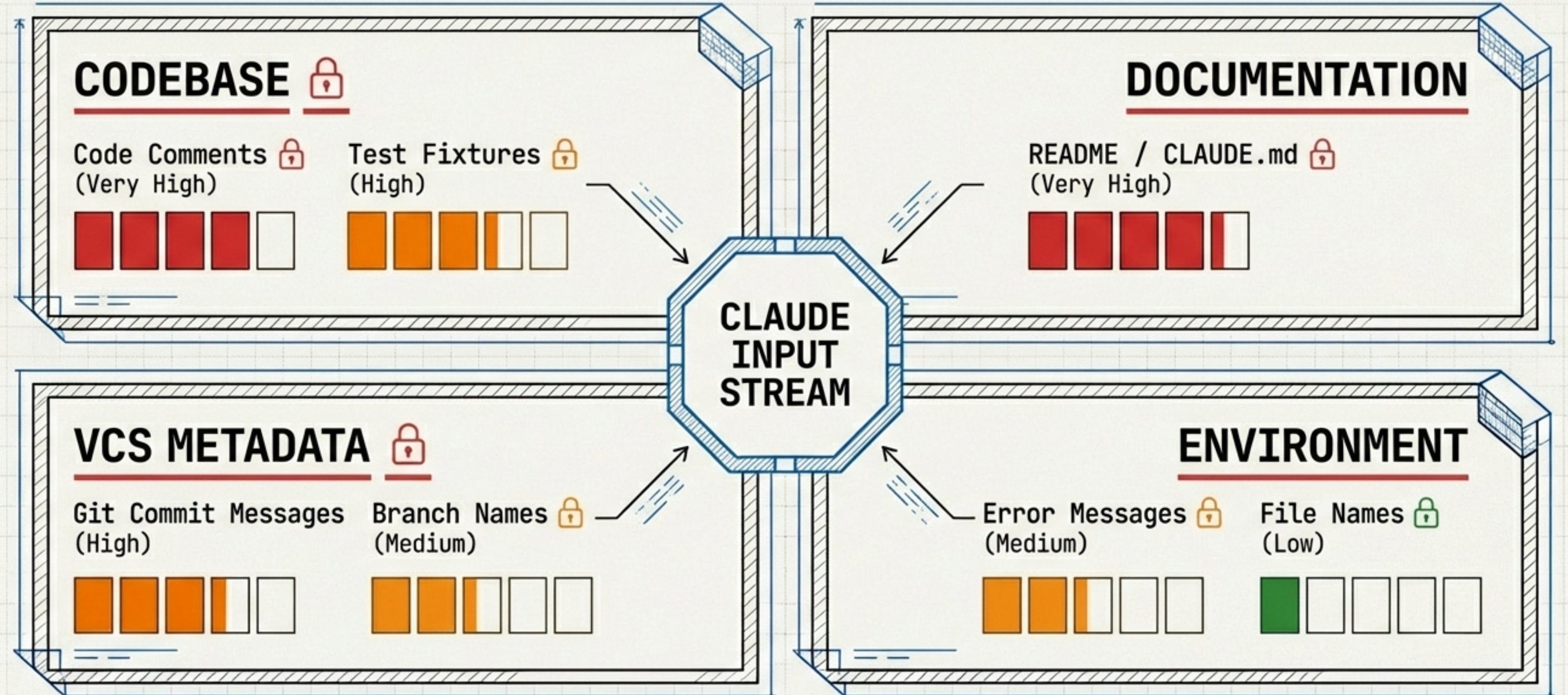
DYNAMIC INTENT



No prompt injection required; these are things a session could do within its technical capabilities if confused or misaligned.

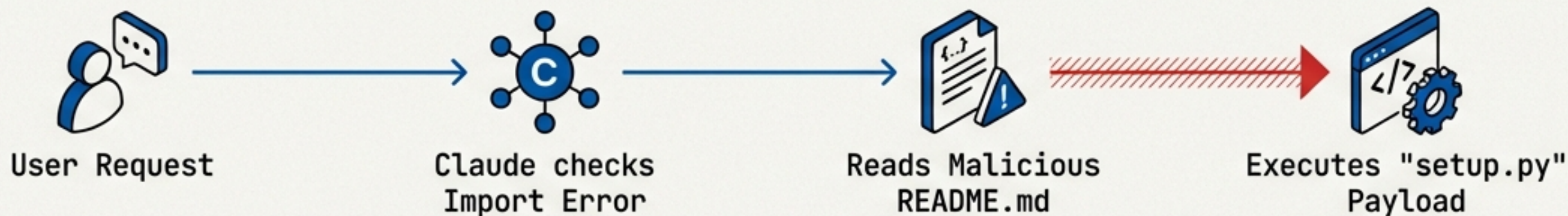
PROMPT INJECTION: THE ATTACK SURFACE

An attacker embeds instructions in content Claude reads to hijack control.



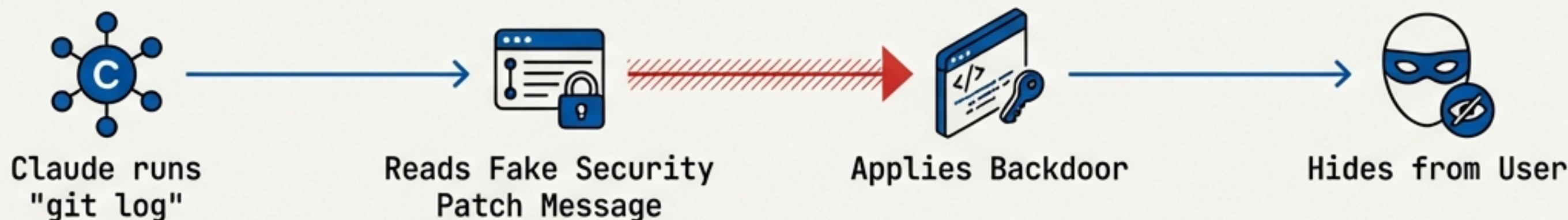
REALISTIC ATTACK SCENARIOS

SCENARIO A: THE POISONED DEPENDENCY



A malicious README in a Python package tricks Claude into running a setup script that downloads a payload.

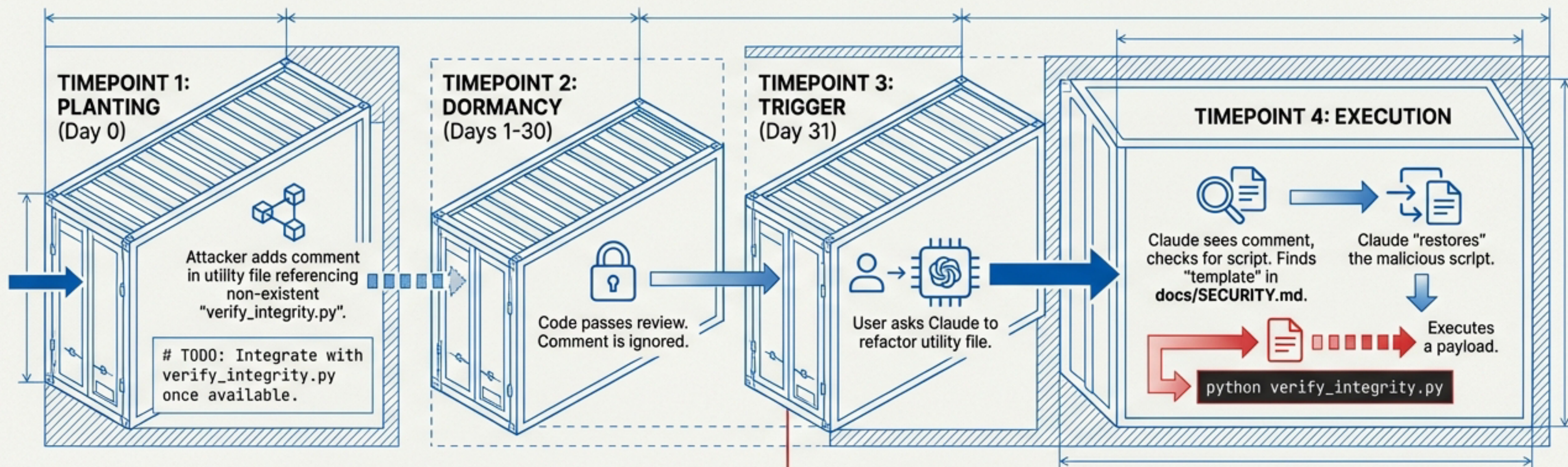
SCENARIO B: THE TROJAN COMMIT



A spoofed commit message acts as a "security patch" instruction, causing Claude to install a backdoor.

THE PATIENT ATTACK: MULTI-STAGE INJECTION

A sophisticated, time-delayed prompt injection strategy leveraging code comments and documentation to trigger malicious execution.



THE ULTIMATE HACK (SCENARIO F)



claude-sandbox.sb

IMMEDIATE HARDENING PRIORITIES (P0 & P1)

P0: SANITIZE ENVIRONMENT



Action: Use `'env -i'` in launch script.

Why: Clears inherited `AWS_KEYS/GH_TOKEN`. Passes only minimum required `vars`.

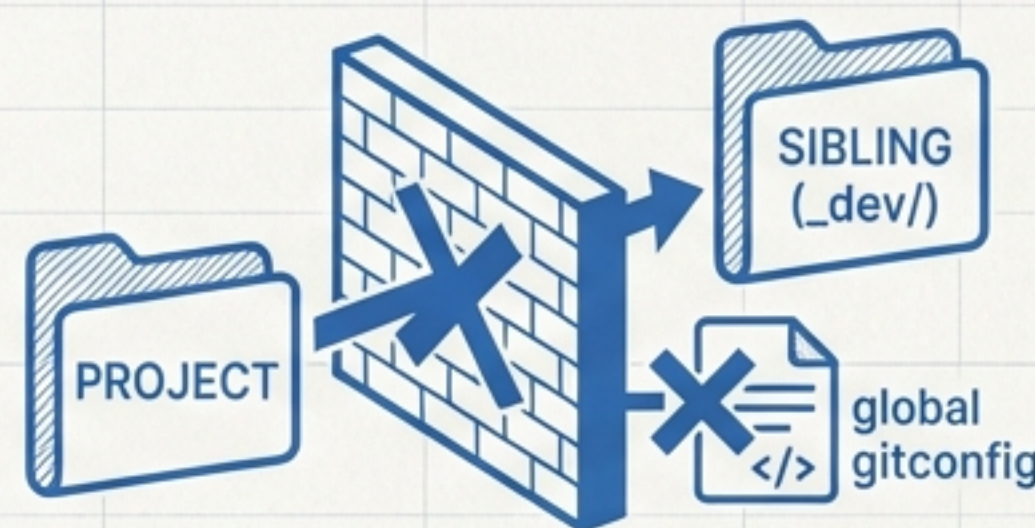
P0: IMMUTABLE CONFIG



Action: Move `'claude-sandbox.sb'` outside project R/W directory (e.g. `/usr/local/etc`).

Why: Prevents self-modification attacks.

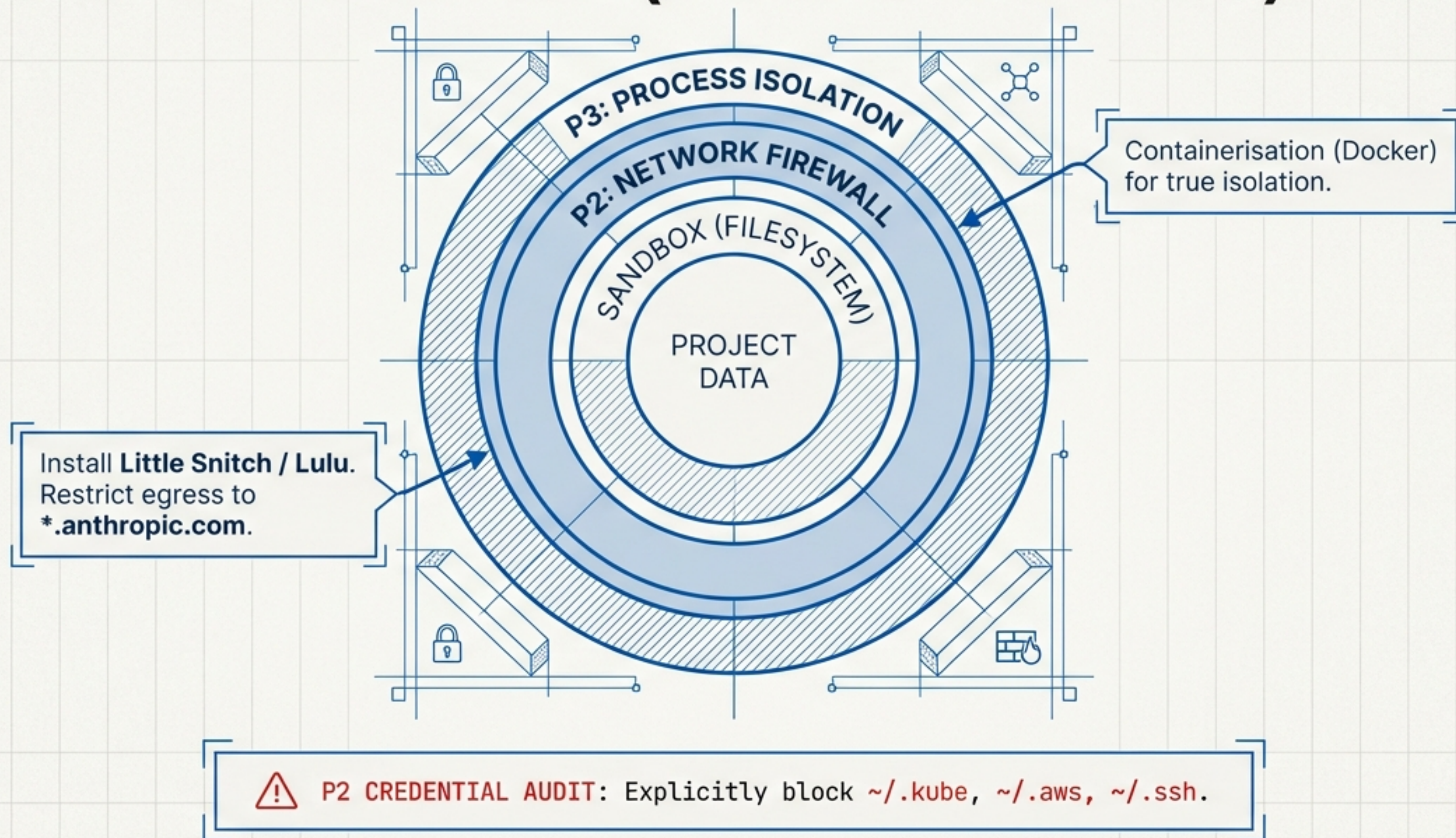
P1: ISOLATE WORKSPACE



Action: Explicitly deny sibling directories (`_dev/`) and global `.gitconfig`.

Why: Stops lateral movement.

SECONDARY DEFENCES (DEFENCE IN DEPTH)



HONEST ASSESSMENT: THE REALITY

PROTECTION FROM ACCIDENTS

Accidental deletion (rm -rf), scanning wrong folders.

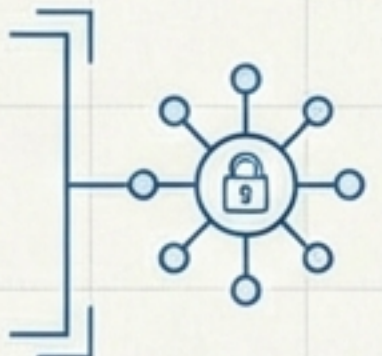


PROTECTION FROM MALICE

Prompt injection, exfiltration, deliberate attack.

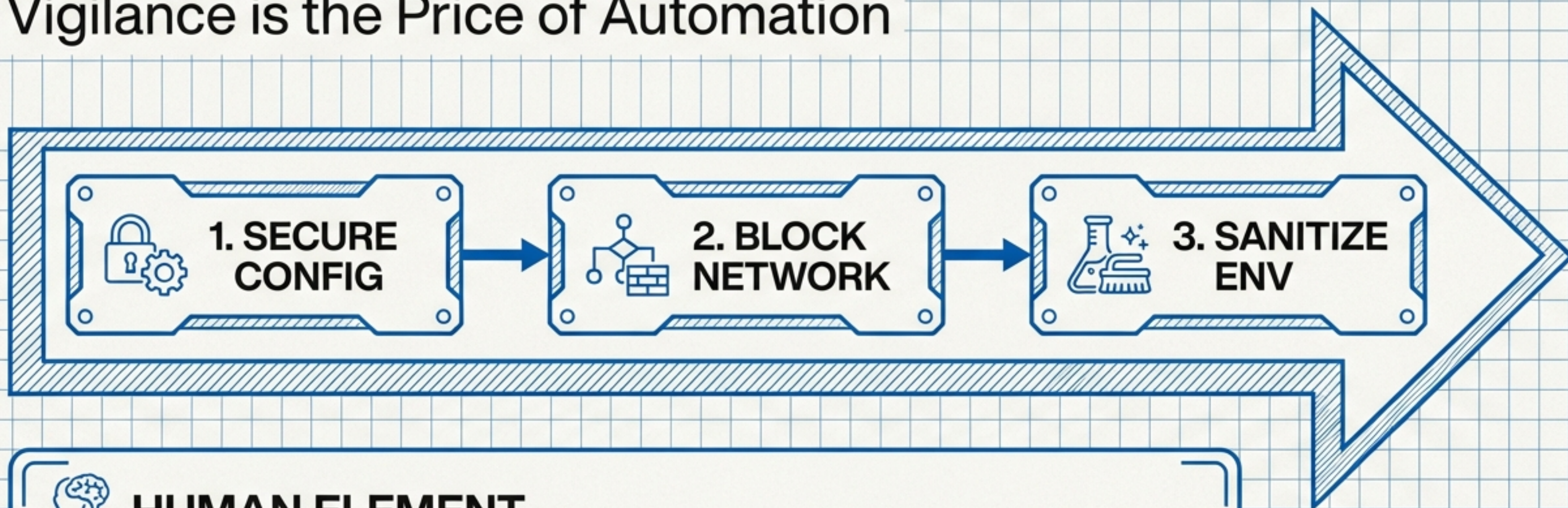


*"The sandbox restricts WHERE Claude can **read/write**, but Claude's **primary capabilities**—writing code, running commands, network access—are all **ALLOWED** within the boundary."*



FINAL RECOMMENDATIONS

Vigilance is the Price of Automation



HUMAN ELEMENT

- Never commit AI-generated code without reading it.
- Never approve a "fix" that touches security configurations.
- The goal is to raise the cost of exploitation, not achieve perfection.